COER UNIVERSITY, ROORKEE

Assignment-3: Software Testing and Quality Assurance

MCA 2nd Year | Subject Code: MCA211D

Name: SRISHTI MALIK Roll No.: 253028081

Q1. What is Test Planning? Explain the main steps involved in the test planning process.

Test Planning is the activity of defining the objectives, scope, approach, resources, schedule and deliverables of a testing effort before testing begins. It answers the questions *what to test, how to test, who will test, when to test and what is needed to test*. The output of test planning is the **Test Plan** document, which acts as a roadmap and a communication tool between the testing team, developers, managers and stakeholders.

Main steps in the test planning process

1. **Analyse the product and requirements:** Study the SRS, design documents and user stories to understand the application, its users and its critical functions.
2. **Define the scope and objectives:** Decide which features/modules will be tested (in scope) and which will not (out of scope), and what the testing is expected to achieve.
3. **Perform risk analysis:** Identify product and project risks (complex modules, new technology, tight deadlines, unstable requirements) and prioritise testing on high-risk areas.
4. **Define the test strategy / approach:** Choose test levels, test types (functional, performance, security, usability, etc.), techniques and manual vs. automation approach.
5. **Define entry and exit criteria:** State the conditions under which testing can start (build ready, environment ready) and when it can stop (e.g., 95% test cases passed, no open critical defects).
6. **Identify resources and roles:** Estimate the people, skills, hardware, software, tools and test data needed and assign responsibilities.
7. **Set up the test environment:** Plan the hardware, OS, browsers, databases, network and tools required to replicate the production environment.
8. **Estimate effort and prepare the schedule:** Estimate time and cost for each activity and prepare a timeline with milestones.
9. **Define test deliverables and metrics:** List the documents to be produced (test cases, bug reports, summary report) and the metrics that will be tracked.
10. **Review and obtain approval:** Get the test plan reviewed and signed off by stakeholders; update it whenever scope or requirements change.

Q2. What is a Test Strategy? Explain its importance in software testing.

A **Test Strategy** is a high-level document that describes the general approach, principles and methods to be used for testing across a project or an entire organisation. It is usually static, derived from the business requirements, and rarely changes during the project. It defines *how* testing will be carried out in broad terms rather than listing individual tests.

Typical contents

- Testing objectives and scope
- Test levels and test types to be performed
- Test techniques and tools (including automation approach)

- Test environment and test data approach
- Defect management and reporting process
- Risk and mitigation approach
- Entry/exit criteria and roles and responsibilities

Importance of a Test Strategy

- **Provides direction:** Gives all team members a common understanding of how testing will be done.
- **Ensures consistency:** The same approach is followed across projects, teams and releases.
- **Risk-based focus:** Directs effort to the most critical and risky areas of the application.
- **Better resource utilisation:** Avoids duplicate or unnecessary testing, saving time and cost.
- **Improves quality:** Systematic coverage reduces the chance of defects escaping to production.
- **Helps in estimation and planning:** Forms the basis from which detailed test plans are prepared.
- **Improves communication:** Stakeholders understand testing scope, limitations and expected outcomes.

Q3. What is a Test Plan? Explain the major contents of a Test Plan.

A **Test Plan** is a detailed document that describes the scope, approach, resources and schedule of intended testing activities for a specific project or release. It identifies the items to be tested, the features to be tested, the testing tasks, who will do each task, and the risks that require contingency planning. IEEE 829 provides the standard structure for test documentation.

Major contents of a Test Plan

Section	Description
Test Plan Identifier	Unique ID, version and title of the document.
Introduction	Purpose of the plan, project background and references (SRS, design docs).
Test Items	The software modules, builds or versions that will be tested.
Features to be Tested	Functional and non-functional features covered by testing.
Features Not to be Tested	Items excluded from testing with reasons.
Approach / Strategy	Test levels, types, techniques, tools and automation plan.
Item Pass/Fail Criteria	Rules for deciding whether a test item has passed or failed.
Suspension and Resumption Criteria	Conditions to pause testing (e.g., blocker defects) and to restart it.
Test Deliverables	Test cases, test data, bug reports, test summary report, metrics.
Test Environment	Hardware, software, network, tools and test data requirements.
Responsibilities and Staffing	Roles of testers, leads, developers and managers; training needs.
Schedule	Milestones, start/end dates and effort estimates.
Risks and Contingencies	Potential risks (late delivery, resource shortage) and mitigation plans.
Approvals	Sign-off by project manager, QA lead and stakeholders.

Q4. Differentiate between a Test Strategy and a Test Plan.

Basis	Test Strategy	Test Plan
Definition	High-level document defining the general testing approach and principles.	Detailed document describing what, when, how and who will test for a specific project/release.
Level	Organisation or product level.	Project or release level.
Scope	Broad and generic; applies to many projects.	Specific to a single project or release.
Nature	Mostly static; changes rarely.	Dynamic; updated as the project progresses.
Prepared by	Test manager / QA head / senior management.	Test lead / test manager.
Derived from	Business requirements and organisational standards.	Test strategy, SRS and project requirements.
Main content	Test levels, types, techniques, tools, defect process, risks.	Scope, items, features, schedule, resources, entry/exit criteria, deliverables.
Focus	*How* testing will be approached.	*What, when, who and how much* will be tested.
Existence	Can exist independently of a project.	Exists as part of a project and refers to the strategy.

Q5. What are the main goals and objectives of software testing? Explain.

Software testing is the process of executing a program with the intent of finding defects and evaluating whether it meets the specified requirements. Its goals and objectives are:

- **Finding defects:** Detect errors, faults and failures as early as possible so that they can be fixed before release.
- **Verifying requirements:** Confirm that the software does what the customer and the specification require (verification and validation).
- **Preventing defects:** Reviewing requirements and design early prevents defects from being introduced into the code.
- **Ensuring quality:** Check reliability, performance, security, usability and compatibility to ensure the product is of acceptable quality.
- **Building confidence:** Provide stakeholders with information and confidence that the system is ready for release.
- **Reducing risk and cost:** Defects found early are far cheaper to correct than those found after deployment.
- **Ensuring compliance:** Verify conformance to legal, regulatory, contractual and industry standards.
- **Providing information for decision-making:** Test reports and metrics help management decide on release readiness.
- **Improving the process:** Defect analysis helps to improve development and testing processes in future projects.

Q6. What are Test Phases? Explain the different phases of software testing.

Test phases are the sequential stages through which testing activities are organised. The term is used in two related senses: the phases of the **Software Testing Life Cycle (STLC)**, and the **levels of testing** performed on the software.

A. Phases of the Software Testing Life Cycle (STLC)

1. **Requirement Analysis:** Testers study requirements to identify testable items and clarify ambiguities. *Output:* requirement traceability matrix, automation feasibility report.
2. **Test Planning:** Define scope, strategy, effort, resources and schedule. *Output:* test plan and effort estimation.
3. **Test Case Design / Development:** Write test cases and scripts, and prepare test data. *Output:* reviewed test cases and test data.
4. **Test Environment Setup:** Prepare hardware, software and network, and run a smoke test to check readiness. *Output:* ready environment.
5. **Test Execution:** Run test cases, compare expected and actual results, and log defects. *Output:* execution status and bug reports.
6. **Test Cycle Closure:** Evaluate exit criteria, analyse metrics, document lessons learned. *Output:* test closure/summary report.

B. Levels (phases) of testing

Level	Description
Unit Testing	Testing of individual functions/classes by developers, usually white-box.
Integration Testing	Testing the interfaces and data flow between combined modules (top-down, bottom-up, big-bang approaches).
System Testing	Testing the complete integrated system against the specified requirements (functional and non-functional).
Acceptance Testing	Testing by users/customers (alpha and beta testing, UAT) to decide whether the system is acceptable for release.

Q7. What factors should be considered while developing a Test Strategy?

A good test strategy must be tailored to the project. The main factors to consider are:

- **Project requirements and scope:** Size, complexity, domain and criticality of the application.
- **Risk:** Identify high-risk modules (financial, safety, security-critical) and give them more testing effort.
- **Type of application:** Web, mobile, desktop, embedded or enterprise systems need different test types and tools.
- **Development model:** Waterfall, V-model, Agile or DevOps affects test timing, documentation and automation.
- **Time and budget constraints:** Deadlines and cost limit the depth of testing that is feasible.
- **Resources and skills:** Availability of testers, their experience, training needs and tool expertise.
- **Test environment and test data:** Availability of realistic environments, devices, browsers and data.
- **Tools and automation:** Which tests should be automated (regression, repetitive) and which should remain manual.

- **Test levels and types:** Functional, performance, security, usability, compatibility and regression requirements.
- **Standards and compliance:** Legal, regulatory and organisational standards that must be satisfied.
- **Entry/exit criteria and defect management:** Quality targets, severity handling and reporting tools.
- **Stakeholder expectations and customer involvement:** Level of user participation and release criteria.
- **Past experience:** Lessons learned and defect history of previous projects or releases.

Q8. Explain the resource requirements needed for software testing.

Resource planning ensures that everything needed to carry out testing is available at the right time. The main resource categories are:

Resource	Details
Human resources	Test manager/lead, test analysts, test engineers, automation engineers, performance/security specialists and domain experts, along with their training needs.
Hardware	Servers, desktops, laptops, mobile devices and network equipment that match the production configuration.
Software	Operating systems, browsers, databases, middleware and the build of the application under test.
Test tools	Test management (Jira, TestRail), automation (Selenium, Appium), performance (JMeter), defect tracking, CI/CD tools.
Test environment	Isolated and stable environment replicating production, with proper configuration and access control.
Test data	Valid, invalid, boundary and volume data; masked production data where needed.
Time	Sufficient schedule for planning, design, execution, retesting and regression cycles.
Budget	Costs of people, licences, infrastructure, cloud/device labs and training.
Documentation	SRS, design documents, templates, test plan, checklists and standards.
Support	Cooperation from developers, administrators, business analysts and customers.

Q9. What is a Test Schedule? Explain the importance of scheduling in software testing.

A **Test Schedule** is a timetable of all testing activities showing their start and end dates, durations, dependencies, milestones and the people responsible. It is usually part of the test plan and is prepared from effort estimates and the overall project plan. Typical items scheduled include test planning, test case design, environment setup, test execution cycles, regression testing, defect retesting, UAT and test closure.

Importance of scheduling

- **Time management:** Ensures that testing is completed within the project deadline.
- **Resource allocation:** Helps assign testers, tools and environments efficiently and avoid conflicts.
- **Tracking progress:** Planned versus actual progress can be monitored through milestones.
- **Early risk identification:** Delays and bottlenecks are detected early so that corrective actions can be taken.
- **Coordination:** Aligns testing with development, deployment and release activities and sets clear dependencies.

- **Prioritisation:** Gives enough time to high-risk and critical areas.
- **Improved communication:** Stakeholders know when builds are needed and when results will be available.
- **Cost control:** Prevents overruns caused by idle time and last-minute rushing.

Q10. What is a Test Case? Explain the main components of a test case.

A **Test Case** is a set of conditions, inputs, execution steps and expected results developed to verify a particular feature or requirement of the software. It tells the tester exactly what to do and what outcome to expect, making testing repeatable and measurable.

Main components of a test case

Component	Description
Test Case ID	Unique identifier (e.g., TC_LOGIN_001).
Title / Description	Short statement of what is being tested.
Module / Feature	Area of the application to which the case belongs.
Requirement ID	Link to the requirement for traceability.
Preconditions	State required before execution (user registered, DB loaded, etc.).
Test Data	Input values to be used.
Test Steps	Ordered actions to be performed.
Expected Result	Behaviour the system should show if it is working correctly.
Actual Result	What actually happened during execution.
Status	Pass / Fail / Blocked / Not Executed.
Priority / Severity	Importance of the test case and impact if it fails.
Postconditions	System state after execution.
Environment, Executed by, Date, Remarks	Execution details and comments.

Example

Field	Value
Test Case ID	TC_LOGIN_001
Description	Verify login with valid credentials
Precondition	User account exists and the login page is open
Test Data	Username: user01, Password: Pass@123
Steps	1. Enter username 2. Enter password 3. Click Login
Expected Result	User is redirected to the dashboard
Status	Pass

Q11. Explain the process of designing and writing effective test cases.

Process

1. **Understand the requirements:** Study the SRS, use cases and user stories; clarify doubts with analysts and developers.
2. **Identify test conditions/scenarios:** Break each requirement into testable conditions (positive, negative, boundary and exceptional).
3. **Select test design techniques:** Apply equivalence partitioning, boundary value analysis, decision tables, state transition testing, error guessing, etc.
4. **Write the test cases:** Document ID, description, preconditions, data, steps and expected results in the standard template.
5. **Prepare test data:** Create valid, invalid and boundary data sets.
6. **Map to requirements:** Maintain a traceability matrix to ensure complete coverage.
7. **Review and refine:** Have peers, leads and developers review for correctness, coverage and clarity.
8. **Prioritise and organise:** Group cases into suites (smoke, sanity, regression) and prioritise by risk.
9. **Maintain:** Update cases as requirements change; remove obsolete or duplicate ones.

Characteristics of an effective test case

- **Clear and simple:** Written in unambiguous language that anyone can follow.
- **Accurate and specific:** Tests one thing, with exact inputs and expected results.
- **Traceable:** Linked to a requirement.
- **Repeatable and independent:** Gives the same result every time and does not depend on other cases.
- **Has a high probability of finding defects:** Covers both positive and negative situations.
- **Reusable and maintainable:** Easy to update and use for regression testing.
- **Economical:** Does not duplicate other test cases.

Q12. What is Bug Reporting? Explain the important information included in a bug report.

Bug Reporting is the process of documenting a defect found during testing and communicating it to the development team in a clear, complete and reproducible manner. A good bug report helps developers locate and fix the problem quickly, and provides a record for tracking and metrics. Defect tracking tools such as Jira, Bugzilla and Mantis are normally used.

Important information in a bug report

Field	Description
Bug ID	Unique identifier assigned by the tracking tool.
Title / Summary	One-line, specific description of the defect.
Description	Detailed explanation of the problem.
Steps to Reproduce	Exact numbered steps that reproduce the defect.
Expected Result	What should have happened.
Actual Result	What actually happened.
Severity	Technical impact (Critical, Major, Minor, Trivial).
Priority	Urgency of fixing (High, Medium, Low).

Field	Description
Status	Current state in the life cycle (New, Open, Fixed, Closed, etc.).
Environment	OS, browser/device, application version/build number, database.
Module / Component	Part of the application affected.
Attachments	Screenshots, screen recordings, logs, stack traces.
Reported by / Assigned to	Tester who found it and the developer responsible.
Date reported	Date and time of logging; useful for defect-age analysis.

Tips for a good bug report: report one defect per report, be objective and precise, make it reproducible, avoid blaming language, check for duplicates first and report as soon as the defect is found.

Q13. Explain the Bug Life Cycle in software testing.

The **Bug (Defect) Life Cycle** is the set of states a defect passes through from the time it is first found until it is closed. It ensures that every defect is tracked, assigned, fixed and verified in a controlled way.

New → Assigned → Open → Fixed → Retest → Verified → Closed

If the fix fails: Retest → Reopened → Assigned (cycle repeats)

Bug states

State	Meaning
New	The tester logs a new defect for the first time.
Assigned	The test/project lead reviews it and assigns it to a developer.
Open	The developer starts analysing and working on the fix.
Fixed	The developer has corrected the code and delivered the fix.
Pending Retest	The fix is released in a new build and waits for the tester.
Retest	The tester re-executes the failed test to confirm the fix.
Verified	The defect no longer occurs and no side effects are found.
Closed	The defect is confirmed resolved and no further action is needed.
Reopened	The defect still exists after the fix; it returns to the developer.
Rejected	The developer decides it is not a valid defect (works as designed).
Duplicate	The same defect has already been reported.
Deferred	The fix is postponed to a later release because of low priority or impact.
Not Reproducible / Cannot Fix	The developer cannot reproduce it, or it cannot be fixed technically.

Q14. What are Test Metrics? Explain their importance in measuring testing activities.

Test Metrics are quantitative measures that describe the status, progress, quality and effectiveness of testing activities and of the product under test. They convert testing information (test cases run, defects found, time

spent) into numbers that can be analysed and compared. Examples are percentage of test cases passed, defect density and defect removal efficiency.

Importance of test metrics

- **Measure progress:** Show how much testing is complete against the plan.
- **Measure quality:** Indicate the stability and quality of the product (defect trends, defect density).
- **Evaluate effectiveness:** Show how well the testing process and the team find defects.
- **Support decision-making:** Help managers decide on release readiness and where to focus effort.
- **Estimation and planning:** Historical metrics improve future effort and schedule estimates.
- **Identify bottlenecks and risks:** Highlight problem modules, delays and resource issues early.
- **Process improvement:** Trends help find weaknesses in development and testing practices.
- **Communication:** Give stakeholders objective, easy-to-understand information instead of opinions.

Q15. Explain different types of testing metrics used in software testing.

Classification

- **Process metrics:** Measure the efficiency of the testing process (e.g., test execution rate, defect removal efficiency, test effort).
- **Product metrics:** Measure the quality of the software (e.g., defect density, defect severity distribution, defects per module).
- **Project metrics:** Measure progress, cost and schedule of the testing project (e.g., planned vs. actual effort, schedule variance, budget variance).

Commonly used test metrics

Metric	Formula / Description	Purpose
Test Case Execution %	(Test cases executed ÷ Total test cases) × 100	Execution progress
Test Case Pass %	(Test cases passed ÷ Test cases executed) × 100	Product quality
Test Case Fail %	(Test cases failed ÷ Test cases executed) × 100	Product quality
Defect Density	Total defects ÷ Size (KLOC or function points)	Code quality
Defect Detection Percentage (DDP)	(Defects found in testing ÷ Total defects found including after release) × 100	Test effectiveness
Defect Removal Efficiency (DRE)	Defects removed before release ÷ (Defects before release + after release) × 100	Effectiveness of the process
Defect Leakage	(Defects found in UAT/production ÷ Defects found in testing) × 100	Missed defects
Requirement / Test Coverage	(Requirements covered by tests ÷ Total requirements) × 100	Completeness
Defect Age	Date closed − Date reported	Fix speed
Defect Severity Index	Distribution of defects by severity	Risk assessment
Automation Coverage	(Automated test cases ÷ Total test cases) × 100	Automation level
Schedule / Effort Variance	(Actual − Planned) ÷ Planned × 100	Project control

Q16. What are Test Case Design Techniques? Explain their importance in software testing.

Test Case Design Techniques are systematic methods used to derive a small but effective set of test cases from requirements, specifications or code. Since exhaustive testing of all possible inputs is impossible, these techniques help select test cases that give maximum coverage and find the maximum number of defects with minimum effort.

Categories

Category	Techniques
Black-box (specification-based)	Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, State Transition Testing, Cause-Effect Graphing, Use Case Testing.
White-box (structure-based)	Statement Coverage, Branch/Decision Coverage, Condition Coverage, Path Coverage, Loop Testing.
Experience-based	Error Guessing, Exploratory Testing, Checklist-based Testing.

Importance

- **Reduces the number of test cases** while still maintaining good coverage.
- **Increases defect detection** by targeting inputs where defects are most likely.
- **Saves time and cost** by avoiding redundant and unnecessary tests.
- **Provides systematic, repeatable coverage** rather than ad-hoc testing.
- **Improves traceability** and objectivity because test cases derive from requirements or code structure.
- **Helps to measure coverage** and decide when to stop testing.
- **Makes test design easier for beginners** by giving a clear procedure to follow.

Q17. What is Equivalence Partitioning? Explain with a suitable example.

Equivalence Partitioning (EP) is a black-box technique in which the input domain is divided into groups (partitions) of data that the software should treat in the same way. If one value from a partition works, all other values in that partition are assumed to work as well, and if one fails, the others are expected to fail too. Therefore only **one representative value from each partition** needs to be tested, which greatly reduces the number of test cases.

Partitions are of two kinds: **valid** (accepted inputs) and **invalid** (rejected inputs).

Example

An application accepts the **age of an applicant between 18 and 60 (inclusive)**. The input domain is divided into three partitions:

Partition	Range	Type	Test value
P1	Age < 18 (0 to 17)	Invalid	10
P2	Age from 18 to 60	Valid	35
P3	Age > 60 (61 and above)	Invalid	75

Only three test cases (10, 35 and 75) are needed instead of testing every possible age. Expected results: 10 → rejected; 35 → accepted; 75 → rejected. Non-numeric or empty input can be added as extra invalid partitions.

Advantages: fewer test cases, good coverage, easy to apply. **Limitation:** does not focus on the edges of the partitions, where many errors occur (hence it is combined with Boundary Value Analysis).

Q18. What is Boundary Value Analysis? Explain with a suitable example.

Boundary Value Analysis (BVA) is a black-box technique based on the observation that errors occur more frequently at the **boundaries** (edges) of input ranges than in the middle. Test cases are designed using values at, just below and just above each boundary (e.g., minimum, min−1, min+1, maximum, max−1, max+1, and a nominal value).

Example

For the same age field that accepts values **from 18 to 60**, the boundaries are 18 (lower) and 60 (upper).

Test value	Position	Expected result
17	Just below the minimum	Invalid – rejected
18	Minimum boundary	Valid – accepted
19	Just above the minimum	Valid – accepted
59	Just below the maximum	Valid – accepted
60	Maximum boundary	Valid – accepted
61	Just above the maximum	Invalid – rejected

Six test cases (17, 18, 19, 59, 60, 61) are enough to check the boundaries. This detects typical off-by-one mistakes such as writing `age > 18` instead of `age >= 18`, or `age < 60` instead of `age <= 60`.

Advantages: very effective at finding boundary-related defects, simple, and fewer test cases than exhaustive testing. **Limitation:** works best for ordered numeric ranges and does not consider combinations of inputs.

Q19. Differentiate between Equivalence Partitioning and Boundary Value Analysis.

Basis	Equivalence Partitioning	Boundary Value Analysis
Concept	Divides inputs into classes that behave the same way.	Tests values at the edges of those classes.
Selection of values	One representative value from each partition (usually from the middle).	Values at, just below and just above each boundary.
Focus	Overall coverage of valid and invalid classes.	Defects located at the extremes of input ranges.
Number of test cases	Fewer.	More than EP for the same input.
Type of defects found	General logic and class-handling errors.	Off-by-one and boundary condition errors (<, ≤, >, ≥).
Example (age 18–60)	10, 35, 75	17, 18, 19, 59, 60, 61
Applicability	Any input type: ranges, sets, types, formats.	Mainly ordered, numeric or range-based inputs.

Basis	Equivalence Partitioning	Boundary Value Analysis
Relationship	Is applied first to identify partitions.	Extension of EP; applied after partitions are identified.
Effort	Lower.	Slightly higher but more thorough at edges.

In practice both techniques are **used together**: EP identifies the partitions, and BVA adds test cases at the boundaries of each partition for better defect detection.

Q20. What is Test Case Planning? Explain the steps involved in test case planning.

Test Case Planning is the activity of planning how test cases will be identified, designed, organised, prioritised, executed and maintained for a project. It decides what test cases are needed, how many, which techniques will be used, who will write and execute them, and when. It ensures that all requirements are covered by well-organised and traceable test cases within the available time and resources.

Steps in test case planning

1. **Study the requirements:** Review the SRS, design and use cases to understand the features to be tested.
2. **Identify test scenarios and conditions:** Decide what has to be verified for each feature, including positive, negative and non-functional aspects.
3. **Decide on the test design techniques:** Choose techniques such as EP, BVA, decision tables and state transition according to the feature.
4. **Define the test case format and standards:** Fix the template, naming convention and the level of detail.
5. **Estimate the number of test cases and effort:** Estimate how many test cases are needed and the time to write and execute them.
6. **Prioritise test cases:** Rank by risk, business importance and frequency of use; identify smoke, sanity and regression sets.
7. **Plan test data and environment:** Identify the data and configuration needed for execution.
8. **Assign responsibilities and schedule:** Allocate modules to testers and set deadlines for design, review and execution.
9. **Plan traceability:** Create a requirement traceability matrix (RTM) to map test cases to requirements.
10. **Plan reviews and approvals:** Arrange peer/lead review and sign-off of the test cases.
11. **Plan maintenance and automation:** Decide which cases will be automated and how the suite will be updated as the software changes.

— End of Assignment —